

AP Examenvragen

Clean question sheet (rephrased, same meaning)

Use this as practice. Write short explanations and draw memory diagrams where requested.

Vraag 1

Consider the following code (exactly as written):

```
#define SUM(a,b) (a b)

inline void print() {
    std::cout << SUM(17, 43);
}
```

1. What does the source look like **after macro expansion**?
2. Does the program compile? If not, explain precisely why.
3. If you replace `inline` by `constexpr`, does anything change in what gets generated/optimized? Explain.
4. What are the practical benefits of `inline`? (Think: ODR/linking, header usage, and optimization.)

Vraag 2

We want a function calculate with `[[nodiscard]]`:

- one overload that returns an `int` computed from $x \cdot y$,
 - one overload that returns a `double` computed from $x - y$.
1. Write the function declarations (and minimal definitions). Does it compile? Explain any issues.
 2. What does `[[nodiscard]]` mean? (Define it.)
 3. What are the benefits of using `[[nodiscard]]`? Give at least two realistic examples.

Vraag 3

Given:

```
std::string str = "Hello";
std::string_view view = str;
str = "Hi";
std::cout << view << "\n";
```

1. **Assume there is no SSO (Small String Optimization).** What is the output *typically*, and what does the C++ standard actually guarantee here? Explain.
2. Explain your answer by drawing the memory situation (stack/heap) before and after the assignment.
3. What is SSO, and how does it optimize memory/performance for short strings?
4. **Assume SSO exists and stores short strings inside the `std::string` object.** What can the output become now? Explain again with a memory diagram.

Vraag 4

Assume this class prints a message in each special member function (constructors/destructor/assignment), and toString() returns "Foo". (Assume correct C++ signatures, e.g. copy constructor takes const Foo& and assignment returns Foo&.)

```
class Foo {
public:
    Foo() { std::cout << "Constructor\n"; }
    ~Foo() { std::cout << "Destructor\n"; }

    Foo(const Foo& other) { std::cout << "Copy constructor\n"; }
    Foo& operator=(const Foo& other) {
        std::cout << "Copy assignment\n";
        return *this;
    }

    std::string toString() const { return "Foo"; }
};

void print(const Foo& f) {
    std::cout << f.toString() << "\n";
}
```

For each code fragment below, write the exact output order and explain which constructor/assignment runs and why (consider copy elision rules in modern C++):

(a)

```
Foo f{};
print(f);
```

(b)

```
Foo f = Foo{};
print(f);
```

(c)

```
Foo f{};
f = Foo{};
print(f);
```

(d) (Same as (b) on purpose.) Is the output identical? Why?

Vraag 5

A type Foo has a default constructor and destructor. All other special member functions are explicitly deleted:

- copy constructor: = delete
- copy assignment: = delete
- move constructor: = delete
- move assignment: = delete

Also assume:

```
Foo getFoo() { return Foo{}; }
```

Answer (compile or not?) and explain:

1. int main(){ Foo f1; Foo f2 = f1; }

```
2. int main(){ Foo f2 = getFoo(); }
```

Vraag 6

A class Foo prints messages in:

- constructor, destructor
- copy constructor and move constructor

Consider:

```
auto f1 = std::make_unique<Foo>();  
auto f2 = std::move(f1);
```

1. What is printed, and in what order?
2. Explain by showing what happens in memory/ownership (what f1 holds before/after `std::move`).
3. Does moving the `unique_ptr` move the Foo object itself? Explain.

Vraag 7

Assume you have Base, Derived, and Final (where `Final final : public Derived`). Each class has:

- a (printing) **virtual destructor**
- virtual methods `foo()` and `bar()` (each prints the class+method name)

Given:

```
Base* b = new Final();  
b->foo();  
b->bar();  
std::cout << "##\n";  
  
Base b2 = *b;    // note: copy into a Base object  
b2.foo();  
b2.bar();  
  
delete b;
```

1. What is the output? Explain every line (dynamic dispatch and object slicing).
2. What changes if the destructor of Base is *not* virtual?
3. How does the compiler/runtime know which `foo()`/`bar()` implementation to call? (High-level explanation.)

Vraag 8

Answer in your own words:

1. What are the dangers of heavy/misused preprocessor includes and macros (e.g. `#include` and `#define`) in large projects?
2. How do we prevent these issues in practice? (Give concrete techniques.)
3. A *FileManager* class does both reading and writing in one class. Explain why that can be a

design problem, and suggest a better design (mention a relevant design principle/pattern).

Vraag 9

Part A (algorithms + lambdas). Consider:

```
std::vector<int> v = {0, 0, 0, 0, 0, 0, 1};

auto print = [=](int element) {
    std::cout << element;
};

auto sum = [=]() {
    int total = 0;
    std::for_each(v.begin(), v.end(), [&](int n) { total += n; });
    return total;
};

std::cout << sum() << "\n";
std::for_each(v.begin(), v.end(), print);
std::cout << "\n";
```

1. What is printed? Explain how the captures work.
2. If you change captures ([=] vs [&]), which behaviors become risky or different? Explain.

Part B (threads + atomics). Consider:

```
std::atomic<int> x{0};

std::thread th1([&]{ x++; });
std::thread th2([&]{ x--; });

th1.join();
th2.join();

std::cout << x << "\n";
```

1. What outputs are possible?
2. Is there a data race? Is there a race condition? Why/why not?

Chapter 10: Generic Programming

1. Why does a **virtual function template** not work in C++? (Explain the core reason.)
2. Consider this idea:

```
struct Text {};

struct TextInterface : Text {
private:
    // ...
public:
    template <typename String>
    void setText(String&& s);
};
```

```
class Reporter {  
public:  
    virtual void report(/* ... */) = 0;  
};
```

Does introducing a non-template Reporter interface solve the original problem? Why or why not?

Laatste vraag

Compare **compile-time polymorphism** (templates, concepts, CRTP, overloading) with **runtime polymorphism** (virtual functions, inheritance).

1. List key benefits of each.
2. List key drawbacks of each.
3. Give one realistic situation where compile-time polymorphism is better, and one where runtime polymorphism is better.

Tip: For memory questions, draw *stack vs heap* and show pointers/ownership. For polymorphism questions, mention vtable/dynamic type and object slicing.